

Intel® RealSense™ SDK：C++ API完全参考手册

· 新一代人工

原创 于 2025-09-07 12:36:20 发布 · 1.1k 阅读 · CC 4.0 BY-SA 版权

Intel® RealSense™ SDK：C++ API完全参考手册

【免费下载链接】librealsense

Intel® RealSense™ SDK

 项目地址: https://gitcode.com/GitHub_Trending/li/librealsense

1. 概述

Intel® RealSense™ SDK（软件开发工具包）为开发者提供了与Intel RealSense深度摄像头交互的全面接口。本手册详细介绍其C++ API的核心组件、类层次结构和使用模式，帮助开发者快速构建深度感知应用。

1.1 核心功能

- **多设备支持**：兼容D400系列、L500系列等RealSense深度摄像头
- **多流处理**：同步获取深度、彩色、红外和IMU（惯性测量单元）数据
- **高级功能**：包括点云生成、帧对齐、距离测量和手势识别
- **跨平台性**：支持Windows、Linux、macOS和Android操作系统

1.2 API 架构



呐不要慌问题大的很

LV1

0

我的组织

0

我的项目

0

我的贡献

进入我的工作台

企业级模型推理API Token



DeepSeek-V3.1-Base

文本生成



252



12486

step3

Step3是阶跃星辰推出的前沿多模态推理模型

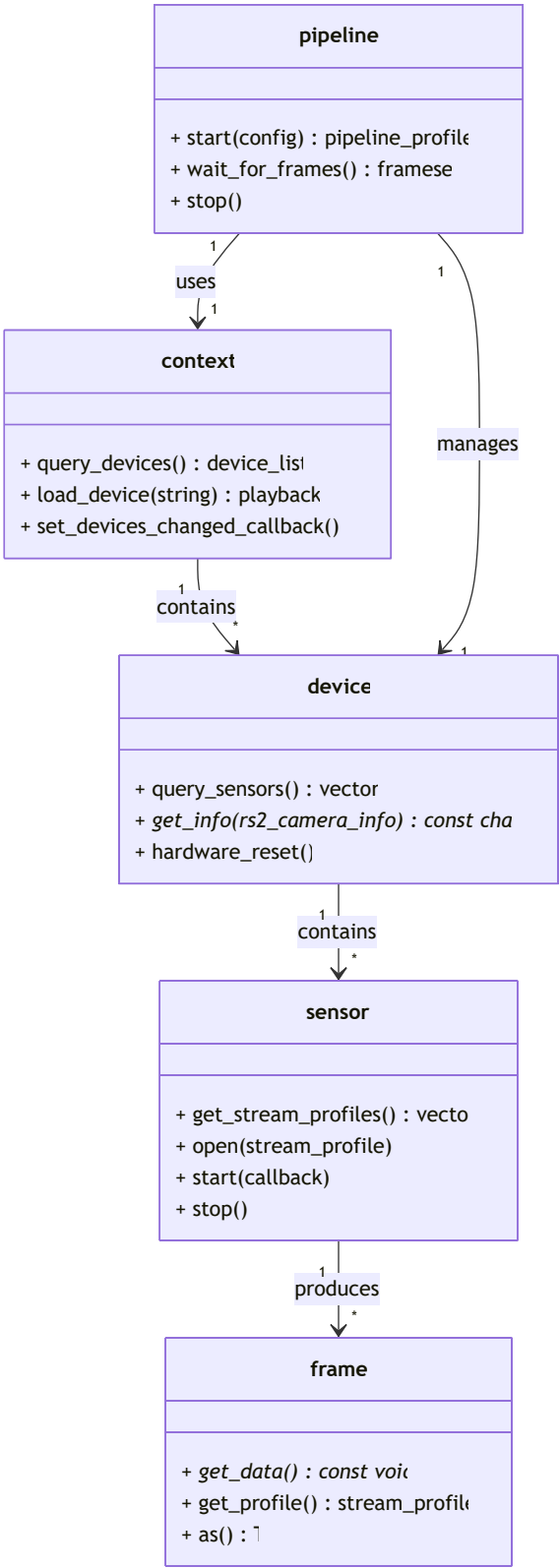
模型使用



克隆



Notebook



2. 核心组件详解

2.1 上下文 (Context)

`rs2::context`是SDK的入口点，负责管理设备枚举和生命周期。

```
1 | #include <librealsense2/rs.hpp>
2 |
3 | // 创建上下文实例
4 | rs2::context ctx;
5 |
6 | // 查询已连接设备
7 | auto devices = ctx.query_devices();
```

```
8 | if (devices.size() == 0) {
9 |     throw std::runtime_error("No RealSense devices found");
10| }
11|
12| // 打印设备信息
13| for (auto&& dev : devices) {
14|     std::cout << "Device: " << dev.get_info(RS2_CAMERA_INFO_NAME) << std::endl;
15|     std::cout << "Serial: " << dev.get_info(RS2_CAMERA_INFO_SERIAL_NUMBER) << std::endl;
16| }
```

收起 ^

关键方法:

- query_devices(): 返回当前连接的设备列表
- load_device(const std::string& file): 从录制文件加载虚拟设备
- set_devices_changed_callback(): 注册设备连接/断开回调

rs2::device 代表物理或虚拟摄像头设备，提供传感器枚举和硬件控制功能。

cpp复制运行

```
1 // 获取第一个设备
2 rs2::device dev = devices[0];
3
4 // 查询设备支持的传感器
5 std::vector<rs2::sensor> sensors = dev.query_sensors();
6
7 // 打印传感器信息
8 for (auto&& sensor : sensors) {
9     std::cout << "Sensor: " << sensor.get_info(RS2_CAMERA_INFO_NAME) << std::endl;
10    std::cout << "Serial: " << sensor.get_info(RS2_CAMERA_INFO_SERIAL_NUMBER) << std::endl;
11    std::cout << "Firmware Version: " << sensor.get_info(RS2_CAMERA_INFO_FIRMWARE_VERSION) << std::endl;
12    std::cout << "Connection Type: " << sensor.get_info(RS2_CAMERA_INFO_CONNECTION_TYPE) << std::endl;
13}
```

展开 v

设备信息查询: | 信息类型 | 描述 | |-----|-----| | RS2_CAMERA_INFO_NAME | 设备名称 | | RS2_CAMERA_INFO_SERIAL_NUMBER | 序列号 | | RS2_CAMERA_INFO_FIRMWARE_VERSION | 固件版本 | | RS2_CAMERA_INFO_CONNECTION_TYPE | 连接类型 (USB/GMSL等) |

2.3 传感器 (Sensor)

rs2::sensor 代表设备上的物理传感器 (如深度传感器、彩色传感器)，负责配置流参数和数据采集。

cpp复制运行

```
1 // 获取深度传感器
2 rs2::depth_sensor depth_sensor = dev.first<rs2::depth_sensor>();
3
4 // 配置深度流
5 rs2::stream_profile depth_profile = depth_sensor.get_stream_profiles()[0];
6 depth_sensor.open(depth_profile);
7
8 // 设置回调函数处理深度帧
9 depth_sensor.start([f](rs2::frame frame) {
10    // 处理深度帧
11})
```

展开 v

常用传感器类型:

- depth_sensor: 深度传感器，提供距离测量能力
- color_sensor: 彩色摄像头，提供RGB图像
- motion_sensor: 运动传感器，提供加速度计和陀螺仪数据

2.4 管道 (Pipeline)

rs2::pipeline提供了简化的数据流管理接口，自动处理设备配置和帧同步。

cpp复制运行

```
1 // 创建管道和配置对象
2 rs2::pipeline pipe;
3 rs2::config cfg;
4
5 // 启用深度流和彩色流
6 cfg.enable_stream(RS2_STREAM_DEPTH, 640, 480, RS2_FORMAT_Z16, 30);
7 cfg.enable_stream(RS2_STREAM_COLOR, 640, 480, RS2_FORMAT_BGR8, 30);
8
9 // 自动配置
```

展开

管道配置选项： | 方法 | 描述 | |-----|-----| | enable_stream() | 启用指定类型、分辨率和格式的流 || enable_device() | 指定使用特定序列号的设备 || enable_record_to_file() | 将流数据录制到文件 || enable_device_from_file() | 从录制文件回放流数据 |

2.5 帧 (Frame)

rs2::frame是所有传感器数据的容器，提供统一的访问接口。

cpp复制运行

```
1 // 处理深度帧
2 void process_depth_frame(rs2::depth_frame depth) {
3     // 获取帧基本信息
4     auto width = depth.get_width();
5     auto height = depth.get_height();
6     auto format = depth.get_profile().format();
7     auto timestamp = depth.get_timestamp();
8
9     std::cout << "Depth frame: " << width << "x" << height
```

展开

常用帧类型：

- depth_frame: 深度帧，包含每个像素的距离信息
- video_frame: 视频帧，包含彩色或红外图像数据
- motion_frame: 运动帧，包含加速度或角速度数据
- composite_frame: 复合帧，包含多个同步的帧

3. 高级功能

3.1 点云生成

cpp复制运行

```
1 // 创建点云对象和颜色对齐器
2 rs2::pointcloud pc;
3 rs2::align align(RS2_STREAM_COLOR);
4
5 // 处理帧集生成点云
6 rs2::frameset frames = pipe.wait_for_frames();
7 auto aligned = align.process(frames);
8
9 rs2::video_frame color = aligned.get_color_frame();
```

展开

3.2 后处理滤镜

cpp复制运行

```
2 rs2::decimation_filter dec_filter; // 降采样滤镜
3 rs2::threshold_filter thr_filter; // 阈值滤镜
4 rs2::disparity_transform depth_to_disparity(true); // 深度转视差
5 rs2::spatial_filter spat_filter; // 空间滤镜
6 rs2::temporal_filter temp_filter; // 时间滤镜
```

```
7 | rs2::disparity_transform disparity_to_depth(false); // 视差转深度
8 |
9 | // 配置滤镜参数
```

展开

常用滤镜类型: | 滤镜 | 作用 | |-----|-----| | decimation_filter | 降低分辨率, 减少计算量 | | threshold_filter | 过滤超出距离范围的像素 | | spatial_filter | 使用邻域像素平滑深度图 | | temporal_filter | 使用时间序列平滑深度图 | | hole_filling_filter | 填充深度图中的孔洞 |

3.3 设备校准

cpp 复制 运行

```
1 | // 检查设备是否支持校准
2 | rs2::device dev = ctx.query_devices()[0];
3 | if (dev.supports(RS2_CAMERA_INFO_CALIBRATION_FILE_VERSION)) {
4 |     std::cout << "Device supports calibration" << std::endl;
5 |
6 |     // 创建校准设备对象
7 |     rs2::calibrated_device calib_dev(dev);
8 |
9 |     // 运行出厂校准重照
```

展开

4. 错误处理与调试

4.1 异常处理

cpp 复制 运行

```
1 | try {
2 |     rs2::context ctx;
3 |     auto devices = ctx.query_devices();
4 |     if (devices.empty()) {
5 |         throw rs2::error("No devices found");
6 |     }
7 |
8 |     rs2::device dev = devices[0];
9 |     rs2::depth_sensor depth_sensor = dev.first<rs2::depth_sensor>();
10 |
11 |     // 尝试打开不支持的分辨率
12 |     rs2::stream_profile invalid_profile = depth_sensor.get_stream_profiles()[0]
13 |         .clone(RS2_STREAM_DEPTH, 0, RS2_FORMAT_Z16);
14 |
15 |     depth_sensor.open(invalid_profile);
16 |     depth_sensor.start([](rs2::frame f) {});
17 | }
18 | catch (const rs2::error& e) {
19 |     std::cerr << "RealSense error calling " << e.get_failed_function() << "(" << e.ge
20 | }
21 | catch (const std::exception& e) {
22 |     std::cerr << "Exception: " << e.what() << std::endl;
23 | }
```

收起

4.2 日志配置

cpp 复制 运行

```
1 | // 配置日志输出到控制台
2 | rs2::log_to_console(RS2_LOG_SEVERITY_INFO);
3 |
4 | // 配置日志输出到文件
5 | rs2::log_to_file(RS2_LOG_SEVERITY_DEBUG, "realsense_log.txt");
6 |
7 | // 启用滚动日志文件 (最大10MB)
8 | rs2::enable_rolling_log_file(10);
```

展开 ∨

5. 应用示例

5.1 实时距离测量

cpp

复制

运行

```
1 #include <librealsense2/rs.hpp>
2 #include <iostream>
3 #include <iomanip>
4
5 int main() {
6     try {
7         // 创建管道和配置
8         rs2::pipeline pipe;
9         rs2::config cfg;
10        cfg.enable_stream(RS2_STREAM_DEPTH, 640, 480, RS2_FORMAT_Z16, 30);
11        cfg.enable_stream(RS2_STREAM_COLOR, 640, 480, RS2_FORMAT_BGR8, 30);
12
13        // 启动管道并获取深度比例
14        auto profile = pipe.start(cfg);
15        float depth_scale = profile.get_device().first<rs2::depth_sensor>().get_depth
16
17        // 创建对齐器（将深度对齐到彩色）
18        rs2::align align(RS2_STREAM_COLOR);
19
20        std::cout << "Real-time Distance Measurement (Press Ctrl+C to exit)" << std::
21        std::cout << "Center distance: ";
22
23        while (true) {
24            // 获取对齐的帧集
25            auto frames = pipe.wait_for_frames();
26            auto aligned_frames = align.process(frames);
27
28            // 获取对齐的深度帧和彩色帧
29            rs2::depth_frame depth = aligned_frames.get_depth_frame();
30            rs2::video_frame color = aligned_frames.get_color_frame();
31
32            if (!depth || !color) continue;
33
34            // 计算中心像素距离
35            int x = color.get_width() / 2;
36            int y = color.get_height() / 2;
37            float distance = depth.get_distance(x, y) * depth_scale;
38
39            // 显示距离（覆盖同一行）
40            std::cout << "\rCenter distance: " << std::fixed << std::setprecision(2)
41                << distance << "m" << std::flush;
42        }
43
44        pipe.stop();
45    }
46    catch (const rs2::error& e) {
47        std::cerr << "RealSense error: " << e.what() << std::endl;
48        return EXIT_FAILURE;
49    }
50    catch (const std::exception& e) {
51        std::cerr << "Error: " << e.what() << std::endl;
52        return EXIT_FAILURE;
53    }
54
55    return EXIT_SUCCESS;
56 }
```

收起 ∧

5.2 录制与回放

cpp

复制

运行

```
1 // 录制数据流到文件
2 void record_stream(const std::string& filename) {
3     rs2::pipeline pipe;
4     rs2::config cfg;
5
6     // 配置要录制的流
7     cfg.enable_stream(RS2_STREAM_DEPTH, 640, 480, RS2_FORMAT_Z16, 30);
8     cfg.enable_stream(RS2_STREAM_COLOR, 640, 480, RS2_FORMAT_BGR8, 30);
9     cfg.enable_record_to_file(filename);
10
11     // 开始录制（录制将自动进行）
12     pipe.start(cfg);
13
14     std::cout << "Recording for 10 seconds..." << std::endl;
15     std::this_thread::sleep_for(std::chrono::seconds(10));
16
17     pipe.stop();
18     std::cout << "Recording saved to " << filename << std::endl;
19 }
20
21 // 从文件回放数据流
22 void playback_stream(const std::string& filename) {
23     rs2::pipeline pipe;
24     rs2::config cfg;
25
26     // 从文件回放
27     cfg.enable_device_from_file(filename);
28     pipe.start(cfg);
29
30     // 获取回放设备
31     rs2::playback playback = pipe.get_active_profile().get_device().as<rs2::playback>
32
33     // 设置回放速度（1.0为正常速度）
34     playback.set_playback_speed(0.5f); // 半速回放
35
36     // 处理回放的帧
37     while (true) {
38         rs2::frameset frames;
39         if (pipe.poll_for_frames(&frames)) {
40             // 获取深度和彩色帧
41             rs2::depth_frame depth = frames.get_depth_frame();
42             rs2::video_frame color = frames.get_color_frame();
43
44             if (depth && color) {
45                 std::cout << "Frame: " << depth.get_frame_number()
46                     << ", Depth resolution: " << depth.get_width() << "x" << de
47                     << ", Color resolution: " << color.get_width() << "x" << co
48             }
49         }
50
51         // 检查回放是否完成
52         if (playback.current_status() == RS2_PLAYBACK_STATUS_STOPPED) {
53             break;
54         }
55     }
56
57     pipe.stop();
58 }
```

收起 ^

6. 性能优化建议

6.1 帧率和分辨率选择

分辨率	帧率	应用场景	典型CPU占用
1280x720	30fps	高细节要求	高
640x480	60fps	实时性要求高	中
424x240	90fps	低延迟要求	低

6.2 多线程处理

cpp复制运行

```
1 // 使用帧队列进行多线程处理
2 rs2::frame_queue depth_queue(10); // 深度帧队列
3 rs2::frame_queue color_queue(10); // 彩色帧队列
4
5 // 生产者线程：获取并分发帧
6 std::thread producer([&]() {
7     rs2::pipeline pipe;
8     pipe.start();
9
10    while (running) {
11        rs2::frameset frames = pipe.wait_for_frames();
12        depth_queue.enqueue(frames.get_depth_frame());
13        color_queue.enqueue(frames.get_color_frame());
14    }
15
16    pipe.stop();
17 });
18
19 // 消费者线程：处理深度帧
20 std::thread depth_consumer([&]() {
21     while (running) {
22         rs2::depth_frame depth;
23         if (depth_queue.poll_for_frame(&depth)) {
24             // 处理深度帧...
25             process_depth(depth);
26         }
27     }
28 });
29
30 // 消费者线程：处理彩色帧
31 std::thread color_consumer([&]() {
32     while (running) {
33         rs2::video_frame color;
34         if (color_queue.poll_for_frame(&color)) {
35             // 处理彩色帧...
36             process_color(color);
37         }
38     }
39 });
40
41 // 等待线程结束
42 producer.join();
43 depth_consumer.join();
44 color_consumer.join();
```

收起 ^

6.3 内存管理

```
1 // 高效处理帧数据
2 void process_frame_data(const rs2::frame& frame) {
3     // 获取帧数据指针和大小
4     const void* data = frame.get_data();
5     size_t size = frame.get_data_size();
6
7     // 避免复制大内存块，直接使用指针操作
```

```
8 |         if (auto depth = frame.as<rs2::depth_frame>()) { 9 |             // 直接处理深度数据
10 |             process_depth_data((const uint16_t*)data, depth.get_width(), depth.get_height(
11 |                 depth.get_stride_in_bytes() / sizeof(uint16_t));
12 |         }
13 |     else if (auto color = frame.as<rs2::video_frame>()) {
14 |         // 直接处理彩色数据
15 |         process_color_data((const uint8_t*)data, color.get_width(), color.get_height(
16 |             color.get_stride_in_bytes());
17 |     }
18 |
19 |     // 帧数据在frame对象销毁时自动释放，无需手动释放
20 | }
```

收起 ^

7. 总结与资源

7.1 关键API速查表

类	核心方法	主要用途
rs2::context	query_devices(), load_device()	设备管理
rs2::sensor	open(), start(), stop()	流控制
rs2::pipeline	start(), wait_for_frames(), stop()	简化数据流管理
rs2::frame	get_data(), get_profile(), as<T>()	帧数据访问
rs2::align	process()	帧对齐
rs2::pointcloud	calculate(), export_to_ply()	点云生成

7.2 进一步学习资源

- **官方文档**: Intel RealSense SDK文档提供了完整的API参考和教程
- **示例代码**: SDK安装包中包含多个示例项目，覆盖各种功能场景
- **GitHub仓库**: https://gitcode.com/GitHub_Trending/li/librealsense
- **社区论坛**: Intel RealSense开发者社区可获取技术支持和最佳实践

7.3 常见问题排查

1. **设备未检测到**: 检查USB连接、权限设置和驱动安装
2. **帧数据为空**: 确保流已正确启用且传感器已启动
3. **性能问题**: 降低分辨率、使用硬件加速或优化算法
4. **校准漂移**: 定期重新校准设备或使用出厂校准重置

通过本手册，开发者可以全面了解Intel® RealSense™ SDK的C++ API，并利用深度摄像头构建各种应用，从简单的距离测量到复杂的三维重建。结合示例代码和最佳实践，能够快速开发出高效、可靠的深度感知应用。

【免费下载链接】librealsense

Intel® RealSense™ SDK

 **项目地址:** https://gitcode.com/GitHub_Trending/li/librealsense

创作声明：本文部分内容由AI辅助生成（AIGC），仅供参考

关于我们 招贤纳士 商务合作 寻求报道 400-660-0108 kefu@csdn.net 在线客服 工作时间 8:30-22:00

公安备案号11010502030143 京ICP备19004658号 京网文〔2020〕1039-165号 经营性网站备案信息
北京互联网违法和不良信息举报中心 家长监护 网络110报警服务 中国互联网举报中心 Chrome商店下载 账号管理规范
版权与免责声明 版权申诉 出版许可证 营业执照 ©1999-2026北京创新乐知网络技术有限公司

